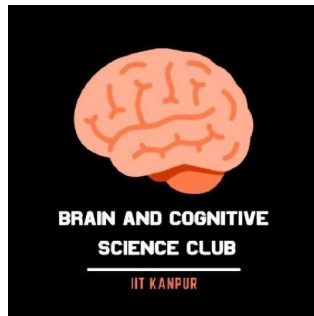




BRAIN AND COGNITIVE SCIENCE CLUB
IIT KANPUR

Judge It Well
Winter Project 2025–26
Final Project Report

February 6, 2026

Abstract

Judge It Well is a project aimed at transforming a general-purpose large language model (LLM) into a specialized legal expert through structured fine-tuning. The goal is to build an AI system capable of interpreting, analyzing, and reasoning through complex legal documents with high accuracy and efficiency.

Mentors and Mentees Overview

Mentors	Mentees
Aditya	Aadya
Poonam	Ankur
Sambhavi	Avinandha
	Chaitanya
	Nandana
	Ojas
	Poorv
	Prashant
	Preet
	Samman
	Suryansh

Contents

1	Introduction	5
2	Libraries	5
2.1	NumPy	5
2.2	Matplotlib	5
2.3	Pandas	5
3	Natural Language Processing	5
3.1	Sentence Segmentation	5
3.2	Tokenisation	5
3.3	Parts of Speech Detection	5
3.4	Lematisation and Stemming	5
3.5	Stop Words Removal	5
3.6	Dependency Parsing	6
3.7	Named Entity Recognition	6
3.8	Coreference Resolution	6
3.9	Representation of Text	6
3.9.1	One-Hot Encoding	6
3.9.2	TF-IDF	6
3.9.3	Bag of Words	7
3.10	Limitations of Classical Methods	7
3.11	Word Embeddings and Contextual Representations	7
3.11.1	Distributed Word Representations	7
3.11.2	Word2Vec	7
3.11.3	CBOW Model	7
3.11.4	GloVe	7
3.11.5	FastText	8
4	Neural Architectures	8
4.1	Linear Regression	8
4.1.1	Model Formulation	8
4.1.2	Gradient-Based Optimization	8
4.1.3	Limitations of Linear Models	8
4.2	Artificial Neural Networks	8
4.2.1	Structure of Neuron	9
4.2.2	Network Architecture	9
4.2.3	Activation Functions	9
4.2.4	Loss Functions	10
4.2.5	Backpropagation	10
4.2.6	Training Procedure	10
4.2.7	Implementation Framework	10
4.2.8	Limitations of Artificial Neural Networks	11
4.3	Sequential Models	11
4.3.1	Sequential Data Representation	11
4.3.2	Recurrent Neural Networks	11
4.3.3	Vanishing Gradient Problem	12
4.3.4	Long Short-Term Memory Networks	12
4.3.5	Next-Word Prediction using LSTM	12
4.3.6	Limitations of Recurrent Models	13

5	Transformer Models and Large Language Models	13
5.1	Encoder–Decoder Architecture	13
5.2	Attention Mechanism	13
5.3	Transformer Architecture	14
5.4	Multi-Head and Masked Attention	14
5.5	Autoregressive Language Modeling	14
5.6	Generative Pre-trained Transformers	14
6	Legal Language and Domain Characteristics	14
6.1	Nature of Legal Text	15
6.2	Challenges in Legal NLP	15
7	Dataset Preparation and Analysis	15
7.1	Dataset Identification	15
7.2	Dataset Structuring	15
7.3	Data Quality Considerations	15
8	Tokenization and Input Processing for LLMs	16
8.1	Subword Tokenization	16
8.2	Tokenizer–Model Alignment	16
8.3	Context Length and Sequence Truncation	16
9	Fine-Tuning LLMs	16
9.1	Full Fine-Tuning	16
9.2	Parameter-Efficient Fine-Tuning	17
9.2.1	Adapters	17
9.2.2	Prompt-Tuning	17
9.2.3	Low-Rank Adaption(LoRA)	17
9.2.4	Quantized Low-Rank Adaption(QLoRA)	17
10	Implementation	18
10.1	Unsloth Library	18
10.2	Hugging Face Integration	18
11	Final Model	18
11.1	Base Model Selection	18
11.2	Legal Domain Adaptation Strategy	18
11.3	Inference Configuration	19
11.4	Generation and Output Handling	19
11.5	Example Usage of the Final Model	19
12	Evaluation and Capabilities	19
12.1	Qualitative Evaluation	19
12.2	Capabilities	19
12.3	Limitations	19
13	Conclusion and Future Scope	20

1 Introduction

Artificial Intelligence has advanced rapidly, yet legal text remains one of the most challenging domains for AI. Legal language is:

- Highly technical
- Context-dependent
- Structured but linguistically unique

General LLMs cannot fully understand or reason through legal documents without domain-specific training. Judge It Well bridges this gap by fine-tuning LLMs on legal data, enabling them to process statutes, contracts, judgments, and legal queries more effectively. In this project we will be working on :

- Fine-tuning an LLM for legal interpretation
- Teaching the model to handle legal reasoning
- Building tools for preprocessing, training, evaluation, and deployment

2 Libraries

2.1 NumPy

NumPy is a Python library used for mathematical computations. It also allows us to work on datatypes like arrays and matrices which are really important for this project.

2.2 Matplotlib

This Python library helps us in plotting graphs to better understand and visualize the relations in our variables.

2.3 Pandas

This Library helps us to read, edit and process large data often in the format of tables which is a crucial part of training neural networks and LLMs.

3 Natural Language Processing

Natural Language Processing, or NLP, is the sub-field of AI that is focused on enabling computers to understand and process human languages.

3.1 Sentence Segmentation

To process language we convert a paragraph to sentences so that the computer can understand each sentence individually rather than the whole paragraph. This can be easily done in python but modern models use complicated method to even do this without properly formatted text.

3.2 Tokenisation

This converts the text into words i.e. Tokens. Tokenization is easy to do in English. We'll just split apart words whenever there's a space between them. And we'll also treat punctuation marks as separate tokens since punctuation also has meaning.

3.3 Parts of Speech Detection

It detects the Part of Speech of each word to further make sense of how the words are related. With this information, we can already start to glean some very basic meaning. For example, we can see that the nouns in the sentence include "London" and "capital", so the sentence is probably talking about London.

3.4 Lemmatisation and Stemming

Stemming is a text preprocessing technique in Natural Language Processing (NLP). Specifically, it is the process of reducing inflected form of a word to one so-called "stem," or root form, also known as a "lemma" in linguistics. It is one of two primary methods—the other being lemmatization—that reduces inflectional variants within a text dataset to one morphological lexeme. In doing so, stemming aims to improve text processing in machine learning and information retrieval systems. But lemmatization is better than stemming because it produces linguistically valid base forms (lemmas) by considering the word's context, part of speech, and morphological structure, ensuring accuracy and meaning preservation.

3.5 Stop Words Removal

We want to consider the importance of each word in the sentence. English has a lot of filler words that appear very frequently like "and", "the", and "a". When doing statistics on text, these words introduce a lot of noise since they appear way more frequently than other words. Some NLP pipelines will flag them as stop words—that is, words that you might want to filter out before doing any statistical analysis.

3.6 Dependency Parsing

The next step is to figure out how all the words in our sentence relate to each other. This is called dependency parsing. But despite a note from the author in 2015 saying that this approach is now standard, it's actually out of date and not even used by the author anymore. In 2016, Google released a new dependency parser called Parsey McParseface which outperformed previous benchmarks using a new deep learning approach which quickly spread throughout the industry. Then a year later, they released an even newer model called ParseySaurus which improved things further. In other words, parsing techniques are still an active area of research and constantly changing and improving.

3.7 Named Entity Recognition

The goal of Named Entity Recognition, or NER, is to detect and label these nouns with the real-world concepts that they represent but NER systems aren't just doing a simple dictionary lookup. Instead, they are using the context of how a word appears in the sentence and a statistical model to guess which type of noun a word represents. A good NER system can tell the difference between "Brooklyn Decker" the person and the place "Brooklyn" using context clues.

3.8 Coreference Resolution

At this point, we already have a useful representation of our sentence. We know the parts of speech for each word, how the words relate to each other and which words are talking about named entities. like in a sentence If we parse it with our NLP pipeline, we'll know that "it" was founded by Romans. But it's a lot more useful to know that "London" was founded by Romans. Coreference resolution is one of the most difficult steps in our pipeline to implement. It's even more difficult than sentence parsing. Recent advances in deep learning have resulted in new approaches that are more accurate, but it isn't perfect yet.

As a human reading this sentence, you can easily figure out that "it" means "London". The goal of coreference resolution is to figure out this same mapping by tracking pronouns across sentences. We want to figure out all the words that are referring to the same entity.

However, we still have one big problem. English is full of pronouns — words like he, she, and it. These are shortcuts that we use instead of writing out names over and over in each sentence. Humans can keep track of what these words represent based on context. But our NLP model doesn't know what pro-

nouns mean because it only examines one sentence at a time.

3.9 Representation of Text

Natural Language Processing (NLP) is a rapidly growing field that focuses on enabling machines to understand and process human language. Text representation is a crucial aspect of NLP that involves converting raw text data into machine-readable form.

Many methods are used to generate text embeddings

3.9.1 One-Hot Encoding

In one hot encoding, every word (even symbols) which are part of the given text data are written in the form of vectors, constituting only of 1 and 0. So one hot vector is a vector whose elements are only 1 and 0. Each word is written or encoded as one hot vector, with each one hot vector being unique. This allows the word to be identified uniquely by its one hot vector and vice versa, that is no two words will have same one hot vector representation. For category

For category $k \in \{1, 2, \dots, K\}$,

$$\mathbf{e}_k(i) = \begin{cases} 1 & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}$$

or as a vector:

$$\mathbf{e}_k = \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

3.9.2 TF-IDF

Term Frequency - Inverse Document Frequency (TF-IDF) is a widely used statistical method in natural language processing and information retrieval. It measures how important a term is within a document relative to a collection of documents (i.e., relative to a corpus).

Words within a text document are transformed into importance numbers by a text vectorization process. There are many different text vectorization scoring schemes, with TF-IDF being one of the most common.

$$\text{TF: } \text{TF}(t, d) = \frac{f_{t,d}}{|d|}$$

$$\text{IDF: } \text{IDF}(t, D) = \log \left(\frac{N}{|\{d \in D: t \in d\}|} \right)$$

TF-IDF:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

3.9.3 Bag of Words

this basically creates a vocabulary of unique words and then counts the frequency of each word. This process is often implemented using tools like CountVectorizer in scikit-learn or Tokenizer in TensorFlow/Keras, which automate the frequency counting and vectorization. The model treats text as a "bag" — ignoring word order and grammar, focusing only on word presence and frequency.

3.10 Limitations of Classical Methods

These classifiers require a numerical input, thus necessitating text to be translated into some kind of vectorial form. At a very high level, text representation techniques practically always begin by indexing different words and creating a vocabulary by which words can be referenced by their index. Before the advent of neural approaches, bodies of text were then transformed into vectors by utilizing relatively simple statistical depictions, the most popularly used being that of Bag-of-Words (BoW). This technique essentially amounts to an unordered word count for vocabulary terms within a document. Most often, these counts are then weighted utilizing frequency terms based on word occurrence statistics, such as Term Frequency (TF) and Inverse Term Frequency (IDF) (Jones, 1972). Through these operations, a single, vectorized representation for each document can be obtained. However, these representations do not contain any real syntactic or semantic information — all sentence ordering information is lost, and the vectors do not encapsulate any real meaning of what they represent.

3.11 Word Embeddings and Contextual Representations

3.11.1 Distributed Word Representations

distributed word representation refers to a method of encoding words as dense, low-dimensional vectors in a continuous vector space, where each dimension captures latent semantic or syntactic features.

3.11.2 Word2Vec

The word2vec model, often referred to as "word to vector," produces dense vector embeddings for words using two main architectures: Skip-gram and CBOW.

The Skip-gram objective maximizes the probability of context words given the center word w_t in a window of size m :

$$P(w_{t+j} | w_t) = \frac{\exp(u_{w_{t+j}}^\top v_{w_t})}{\sum_{w=1}^{|V|} \exp(u_w^\top v_{w_t})} \quad (1)$$

$$\ell(\theta) = \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t) \quad (2)$$

The loss to minimize for a pair (center c , context o) is:

$$J(\theta) = -u_o^\top v_c + \log \sum_{w=1}^{|V|} \exp(u_w^\top v_c) \quad (3)$$

Gradients:

$$\frac{\partial J}{\partial v_c} = -u_o + \sum_{w=1}^{|V|} P(w | c) u_w \quad (4)$$

$$\frac{\partial J}{\partial u_o} = -v_c \quad (5)$$

3.11.3 CBOW Model

CBOW predicts the center w_c from averaged context vectors $\bar{v}_o = \frac{1}{2m} \sum_{-m \leq j \leq m, j \neq 0} v_{w_{t+j}}$:

$$P(w_c | o) = \frac{\exp(u_{w_c}^\top \bar{v}_o)}{\sum_{w=1}^{|V|} \exp(u_w^\top \bar{v}_o)} \quad (6)$$

The loss for a pair:

$$J(\theta) = -u_c^\top \bar{v}_o + \log \sum_{w=1}^{|V|} \exp(u_w^\top \bar{v}_o) \quad (7)$$

Gradient w.r.t. a context vector v_{o_i} :

$$\frac{\partial J}{\partial v_{o_i}} = \frac{1}{2m} \left(-u_c + \sum_{w=1}^{|V|} P(w | o) u_w \right) \quad (8)$$

3.11.4 GloVe

GloVe stands for Global Vectors for Word Representation, an unsupervised algorithm for learning word embeddings from a large text corpus. Unlike word2vec's predictive approach (CBOW/Skip-gram), GloVe factorizes a global word co-occurrence matrix to capture semantic relationships via statistics. GloVe loss function:

$$J = \sum_{i=1}^V \sum_{j=1}^V f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (9)$$

$$f(x) = \begin{cases} \left(\frac{x}{x_{\max}} \right)^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases} \quad (10)$$

3.11.5 FastText

FastText is an extension of word2vec embeddings from Facebook AI Research (FAIR), using subword (character n-gram) information to create robust word vectors. `article amsmath amssymb`

FastText Subword Embeddings:

FastText represents a word w as the average of subword vectors (n-grams of length 3–6, plus the word itself):

$$\mathbf{v}_w = \frac{1}{m} \sum_{g \in G_w} \mathbf{v}_g \quad (11)$$

where G_w is the set of subword n-grams for w , $m = |G_w|$, and $\mathbf{v}_g$ is the embedding for n-gram g .

Like word2vec Skip-gram, it maximizes:

$$J = -\log \sigma(\mathbf{v}_{\text{target}}^\top \mathbf{v}_{\text{context}}) - \sum_{k=1}^K \mathbb{E}_{\text{neg}} [\log \sigma(-\mathbf{v}_{\text{neg}}^\top \mathbf{v}_{\text{context}})] \quad (12)$$

using negative sampling, where $\mathbf{v}_{\text{context}}$ uses subword-averaged vectors.

4 Neural Architectures

4.1 Linear Regression

4.1.1 Model Formulation

Linear regression is a type of supervised machine-learning algorithm that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets. It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

Mathematically, given an input feature vector $\mathbf{x} \in \mathbb{R}^n$, the predicted output $\hat{y}$ is expressed as

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

where $\mathbf{w}$ denotes the weight vector and b represents the bias term. The main goal of linear regression is to find the best-fit line through a set of data points that minimizes the difference between the actual values and predicted values.

4.1.2 Gradient-Based Optimization

The error between actual and predicted values can be quantified using a loss function of the cost function. The cost function takes the model's predicted values and actual values and returns a single scalar value that represents the cost of the model's prediction. Our main goal is to minimize the cost function.

For regression tasks, the Mean Squared Error (MSE) loss is commonly used and is given by

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where y_i represents the true output and $\hat{y}_i$ represents the predicted output for the i -th data point.

The optimization of the loss function is performed using gradient-based methods. Model parameters are updated iteratively by moving in the direction opposite to the gradient of the loss with respect to the parameters. The learning rate η controls the step size of each parameter update. The update rules for the parameters are given by

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

An appropriate choice of learning rate is essential for stable and efficient convergence.

4.1.3 Limitations of Linear Models

Although linear regression provides a simple and interpretable model, it is limited by its assumption of linearity and is unable to capture complex, non-linear relationships present in real-world data. These limitations motivate the introduction of non-linear models, leading to artificial neural networks.

4.2 Artificial Neural Networks

Artificial Neural Networks (ANNs) are computer systems designed to mimic how the human brain processes information. Just like the brain uses neurons to process data and make decisions, ANNs use artificial neurons to analyze data, identify patterns and make predictions. These networks consist of layers of interconnected neurons that work together to solve complex problems. The key idea is that ANNs can “learn” from the data they process, just as our brain learns from experience.

4.2.1 Structure of Neuron

The basic building block of an ANN is the artificial neuron, also known as a perceptron. Given an input vector

$$\mathbf{x} = [x_1, x_2, \dots, x_n],$$

the neuron computes a weighted sum and adds a bias term:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

where $\mathbf{w}$ is the weight vector and b is the bias. The bias allows shifting of the activation threshold independently of the input values.

A non-linear activation function $f(\cdot)$ is then applied:

$$a = f(z)$$

This non-linearity is critical because it enables the network to model complex, non-linear relationships in data. Without non-linear activation functions, stacking multiple layers would result in the entire network behaving as an equivalent single linear transformation, regardless of the number of layers. Consequently, the network would be unable to learn complex patterns, significantly limiting its expressive power.

4.2.2 Network Architecture

An artificial neural network is constructed by arranging artificial neurons into a structured set of layers. These layers typically include an input layer, one or more hidden layers, and an output layer. Each layer performs a transformation on its input and passes the result to the subsequent layer.

Let the input feature vector be denoted by

$$\mathbf{x} \in \mathbb{R}^n.$$

The input layer directly receives this vector without performing any computation. The transformation begins in the first hidden layer.

For a neural network with one hidden layer, the hidden layer activation is computed as

$$\mathbf{h} = f(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}),$$

where $W^{(1)} \in \mathbb{R}^{m \times n}$ is the weight matrix connecting the input layer to the hidden layer, $\mathbf{b}^{(1)} \in \mathbb{R}^m$ is the bias vector, and $f(\cdot)$ is the activation function applied element-wise.

The output layer then computes the final prediction as

$$\hat{\mathbf{y}} = g(W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}),$$

where $W^{(2)} \in \mathbb{R}^{k \times m}$ and $\mathbf{b}^{(2)} \in \mathbb{R}^k$ are the weight matrix and bias vector of the output layer, respectively, and $g(\cdot)$ denotes the output activation function.

In the case of multiple hidden layers, this process generalizes to

$$\mathbf{h}^{(l)} = f(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}),$$

where $\mathbf{h}^{(l)}$ represents the activations of the l -th hidden layer. Stacking multiple layers allows the network to learn hierarchical feature representations, with deeper layers capturing increasingly abstract patterns.

In fully connected neural networks, each neuron in a layer is connected to every neuron in the subsequent layer. This dense connectivity enables the network to learn complex interactions between input features but also increases the number of trainable parameters.

The depth (number of layers) and width (number of neurons per layer) of the network are key architectural hyperparameters that influence the model's representational capacity and computational complexity.

4.2.3 Activation Functions

Activation functions introduce non-linearity into neural networks, enabling them to model complex relationships in data. Without activation functions, stacked layers would reduce to a single linear transformation, limiting the expressive capacity of the network.

The **Rectified Linear Unit (ReLU)**, defined as

$$f(x) = \max(0, x),$$

is widely used in the hidden layers of feedforward neural networks due to its computational efficiency and its ability to mitigate the vanishing gradient problem.

The **sigmoid activation function**,

$$f(x) = \frac{1}{1 + e^{-x}},$$

maps input values to the range $(0, 1)$ and is commonly used in binary classification and gating mechanisms, although it is less suitable for deep networks because of vanishing gradients.

The **hyperbolic tangent function**,

$$f(x) = \tanh(x),$$

maps inputs to the range $(-1, 1)$ and has been widely used in recurrent neural networks and long short-term memory architectures due to its zero-centered output.

4.2.4 Loss Functions

As introduced in the linear regression setting, loss functions quantify the error between model predictions and ground-truth targets and define the optimization objective. In artificial neural networks, the same principle applies; however, the loss is now computed over outputs produced by multiple layers of non-linear transformations.

For regression tasks, Mean Squared Error continues to be a suitable choice, while for classification tasks, loss functions such as cross-entropy are commonly employed. The selected loss function directly influences the gradients propagated through the network during training and therefore plays a crucial role in learning dynamics.

For classification tasks, the Cross-Entropy loss is widely used. In the binary classification setting, it is given by

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

The choice of loss function depends on the nature of the learning task and directly influences the gradients computed during training.

4.2.5 Backpropagation

Backpropagation is the learning algorithm used to train artificial neural networks by efficiently computing the gradients of the loss function with respect to all trainable parameters in the network. It generalizes the gradient-based optimization used in linear regression to multi-layer, non-linear models.

Consider a feedforward neural network with L layers. Let $\mathbf{a}^{(l)}$ denote the activation vector of the l -th layer, $\mathbf{z}^{(l)}$ the corresponding pre-activation vector, $W^{(l)}$ the weight matrix, and $\mathbf{b}^{(l)}$ the bias vector. The forward propagation equations are given by

$$\begin{aligned} \mathbf{z}^{(l)} &= W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \\ \mathbf{a}^{(l)} &= f^{(l)}(\mathbf{z}^{(l)}), \end{aligned}$$

where $f^{(l)}(\cdot)$ denotes the activation function of layer l and $\mathbf{a}^{(0)} = \mathbf{x}$ is the input vector.

Let $\mathcal{L}(\mathbf{a}^{(L)}, \mathbf{y})$ denote the loss function defined at the output layer, as introduced earlier. The goal of backpropagation is to compute the gradient $\frac{\partial \mathcal{L}}{\partial W^{(l)}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$ for all layers $l = 1, \dots, L$.

Backpropagation proceeds by defining the error term (also called the delta) for each layer:

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}.$$

For the output layer L , the error term is computed as

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot f'^{(L)}(\mathbf{z}^{(L)}),$$

where $\odot$ denotes element-wise multiplication and $f'^{(L)}(\cdot)$ is the derivative of the output activation function.

For hidden layers $l = L - 1, \dots, 1$, the error term is recursively computed using the chain rule:

$$\delta^{(l)} = \left(W^{(l+1)\top} \delta^{(l+1)} \right) \odot f'^{(l)}(\mathbf{z}^{(l)}).$$

Once the error terms are obtained, the gradients of the loss function with respect to the parameters are given by

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W^{(l)}} &= \delta^{(l)} \mathbf{a}^{(l-1)\top}, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} &= \delta^{(l)}. \end{aligned}$$

These gradients are then used to update the parameters via gradient descent:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}},$$

where η is the learning rate.

Through repeated application of forward propagation and backpropagation over multiple training epochs, the network parameters are iteratively adjusted to minimize the loss function. Backpropagation enables efficient training of deep neural networks by reusing intermediate computations and ensuring that gradients are computed with computational complexity linear in the number of parameters.

4.2.6 Training Procedure

The training procedure involves iteratively updating the network parameters using gradient-based optimization. For each training epoch, a forward pass is performed to compute the model output, followed by loss evaluation. Backpropagation is then applied to compute gradients with respect to model parameters, which are updated accordingly. This process is repeated until convergence or a predefined number of epochs is reached.

4.2.7 Implementation Framework

The implementation of the artificial neural network was carried out using a combination of scikit-learn and PyTorch. **Scikit-learn** was used for dataset pre-processing, including train-test splitting and feature scaling, while **PyTorch** was employed to define the

neural network architecture, perform backpropagation, and optimize model parameters using gradient-based methods.

Data Preprocessing

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Model Definition

```
import torch.nn as nn

class ANN(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.fc1 = nn.Linear(input_size, 20)
        self.fc2 = nn.Linear(20, 10)
        self.fc3 = nn.Linear(10, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x
```

Training Setup

```
input_size = X_train.shape[1]
model = ANN(input_size)
criterion = nn.BCELoss() # Binary Cross Entropy Loss
optimizer = optim.Adam(model.parameters(), lr=0.001)
# Convert arrays to tensors
inputs = torch.tensor(X_train, dtype=torch.float32)
labels = torch.tensor(y_train, dtype=torch.float32)
epochs = 800
for epoch in range(epochs):

    # Forward pass
    outputs = model(inputs)
    loss = criterion(outputs, labels.unsqueeze(1))

    # Backward pass and optimization
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch+1)%10 == 0:
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {
            loss.item():.4f}")
```

4.2.8 Limitations of Artificial Neural Networks

Despite their ability to model complex non-linear relationships, artificial neural networks operate on

fixed-size input representations and process data without an inherent notion of order or temporal dependency. As a result, they are not well-suited for tasks involving sequential or time-dependent data, such as natural language, where context across positions plays a crucial role.

Additionally, feedforward neural networks do not retain information from previous inputs, limiting their capacity to model long-range dependencies. These limitations motivate the use of sequence-aware architectures, such as recurrent neural networks and long short-term memory models, which explicitly incorporate mechanisms for handling temporal structure.

4.3 Sequential Models

Sequential models are designed to process data where the order of inputs is significant. Unlike feedforward neural networks, which operate on fixed-size inputs independently, sequential models explicitly account for dependencies across time steps, making them suitable for tasks involving ordered data such as text and time series.

4.3.1 Sequential Data Representation

In sequential data, inputs are arranged in a specific order, and the meaning of each element may depend on preceding elements. Such data exhibits temporal dependencies, where information from earlier time steps influences the interpretation of later inputs.

Examples of sequential data include sentences in natural language, where word order determines meaning, and time-series signals, where past observations affect future behavior. Modeling these dependencies requires architectures that can retain and update contextual information across time steps.

4.3.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by explicitly modeling temporal dependencies across time steps. Unlike feedforward neural networks, RNNs maintain a hidden state that allows information from previous inputs to influence the processing of the current input.

Given an input sequence $\{x^{(1)}, x^{(2)}, \dots, x^{(T)}\}$, the RNN processes the sequence iteratively. At each time step t , the hidden state is updated using the current input and the hidden state from the previous time step according to

$$a^{(t)} = g_1 \left(W_{aa} a^{(t-1)} + W_{ax} x^{(t)} + b_a \right),$$

where $a^{(t)}$ denotes the hidden state at time step t , W_{aa} is the recurrent weight matrix, W_{ax} is the input-to-hidden weight matrix, b_a is the bias term, and $g_1(\cdot)$ is a non-linear activation function.

The output at each time step is computed as

$$y^{(t)} = g_2(W_{ya}a^{(t)} + b_y),$$

where W_{ya} is the hidden-to-output weight matrix, b_y is the output bias, and $g_2(\cdot)$ denotes the output activation function.

A key characteristic of recurrent neural networks is temporal parameter sharing. The same set of parameters $\{W_{ax}, W_{aa}, W_{ya}, b_a, b_y\}$ is reused at every time step, allowing the model to generalize across sequences of varying lengths while maintaining consistent transformations throughout the sequence.

By propagating the hidden state across time, RNNs are able to capture short-term temporal dependencies in sequential data. However, when trained on long sequences, standard RNNs face challenges in preserving information over extended time horizons, motivating further analysis of their training dynamics.

Loss Function In recurrent neural networks, predictions are generated at multiple time steps. The total loss over a sequence is defined as the sum of losses computed at each time step. For a sequence of length T_y , the overall loss is given by

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}(\hat{y}^{(t)}, y^{(t)}),$$

where $\hat{y}^{(t)}$ and $y^{(t)}$ denote the predicted and true outputs at time step t , respectively.

Backpropagation Through Time Training recurrent neural networks involves backpropagation through time (BPTT), where gradients are computed by unrolling the network across time steps. At a given time horizon T , the gradient of the loss with respect to a shared weight matrix W is expressed as

$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^T \left. \frac{\partial \mathcal{L}^{(T)}}{\partial W} \right|_{(t)}.$$

4.3.3 Vanishing Gradient Problem

During backpropagation through time, gradients are propagated backward across many time steps. Since this process involves repeated multiplication by

weight matrices and activation derivatives, the gradient values can become very small as they move further back in time.

As a result, parameter updates corresponding to earlier time steps diminish significantly, making it difficult for recurrent neural networks to learn long-range dependencies. This phenomenon is known as the vanishing gradient problem and is a key limitation of standard RNNs when modeling long sequences.

4.3.4 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks were introduced to overcome the vanishing gradient problem observed in standard recurrent neural networks. Unlike traditional RNNs, LSTMs maintain a cell state that enables information to persist across long time horizons, allowing the model to capture long-range dependencies in sequential data.

At each time step t , an LSTM maintains a hidden state h_t and a cell state c_t . Information flow is regulated through gating mechanisms. The forget gate, input gate, and output gate are defined as

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

where $\sigma(\cdot)$ denotes the sigmoid activation function.

A candidate cell state is computed as

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c).$$

The cell state is then updated according to

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

and the hidden state is obtained as

$$h_t = o_t \odot \tanh(c_t).$$

Through controlled information flow and additive state updates, LSTM networks mitigate gradient decay during backpropagation through time, enabling stable learning over long sequences. This makes LSTMs well-suited for sequence modeling tasks such as next-word prediction and language modeling.

4.3.5 Next-Word Prediction using LSTM

An LSTM-based architecture was employed to model next-word prediction on sequential text data. Given

an input sequence of tokens, the model learns to predict the next token in a sequence using the contextual information captured from previous time steps.

Model Definition

```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, 100,
                                       padding_idx=0)
        self.lstm = nn.LSTM(100, 200, batch_first=True)
        self.fc = nn.Linear(200, vocab_size)

    def forward(self, x):
        embedded = self.embedding(x)
        intermediate_hidden_states, (final_hidden_state,
                                     final_cell_state) = self.lstm(embedded)
        output = self.fc(final_hidden_state.squeeze(0))
        return output
```

4.3.6 Limitations of Recurrent Models

Although recurrent neural networks and LSTM-based architectures improve upon feedforward models by capturing sequential dependencies, they still exhibit fundamental limitations when applied to large-scale and long-context sequence modeling tasks.

Recurrent models process sequences sequentially, which restricts parallelization during training and inference. This sequential nature leads to increased computational time and limits scalability on large datasets. Additionally, despite mitigating vanishing gradients, LSTMs struggle to efficiently capture very long-range dependencies, as information must still propagate through multiple recurrent steps.

These limitations become particularly significant in natural language processing tasks involving long documents and complex contextual relationships. As a result, alternative architectures that enable parallel processing and direct modeling of long-range dependencies are required. This motivates the transition to transformer-based models and large language models, which address these challenges through attention mechanisms.

5 Transformer Models and Large Language Models

5.1 Encoder–Decoder Architecture

The encoder–decoder framework is a foundational sequence-to-sequence architecture used in early neural machine translation models. The encoder reads

an input sequence

$$x_1, x_2, \dots, x_T$$

and produces a sequence of hidden states by recursively updating

$$h_i = \text{EncoderRNN}(h_{i-1}, x_i),$$

resulting in hidden representations

$$h_1, h_2, \dots, h_T.$$

The decoder initializes its hidden state using the final encoder state,

$$s_0 = h_T,$$

and generates outputs sequentially according to

$$s_t = \text{DecoderRNN}(s_{t-1}, y_{t-1}).$$

A major limitation of this approach is that the entire input sequence is compressed into a single fixed-length vector h_T . For long sequences, this creates an information bottleneck, leading to loss of contextual information.

5.2 Attention Mechanism

The attention mechanism was introduced to address the fixed-length bottleneck of encoder–decoder models. Instead of relying solely on the final encoder state, attention allows the decoder to dynamically focus on different parts of the input sequence at each decoding step. At decoder time step t , alignment scores are computed between the decoder state and each encoder hidden state:

$$e_{t,i} = \text{score}(s_{t-1}, h_i).$$

These scores are normalized to obtain attention weights:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^T \exp(e_{t,k})}.$$

A context vector is then computed as a weighted sum of encoder states:

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i.$$

The decoder state update incorporates this context:

$$s_t = \text{RNN}(s_{t-1}, y_{t-1}, c_{t-1}),$$

and the output distribution is given by

$$P(y_t | y_{<t}, x) = \text{softmax}(V[s_t; c_t] + b).$$

Although attention improves alignment and long-range dependency modeling, encoder and decoder states must still be computed sequentially, limiting parallelization.

5.3 Transformer Architecture

Transformers remove recurrence entirely and rely solely on self-attention mechanisms to model dependencies within a sequence. Let

$$h_1, h_2, \dots, h_T$$

denote the input token representations. Self-attention computes pairwise relevance between tokens:

$$\alpha_{ij} = \text{softmax}(f(h_i, h_j)),$$

where each token representation is updated as

$$s_i = \sum_{j=1}^T \alpha_{ij} h_j.$$

To compute attention efficiently, token representations are projected into query, key, and value spaces:

$$q_i = W_Q h_i, \quad k_i = W_K h_i, \quad v_i = W_V h_i.$$

Interpretation of Query, Key, and Value Vectors

- The query vector q_i represents what information token i is looking for.
- The key vector k_j represents what kind of information token j contains.
- The value vector v_j represents the actual information content that token j can contribute.

The attention score function is defined as

$$f(h_i, h_j) = q_i^\top k_j,$$

and the normalized attention weights become

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j)}{\sum_{m=1}^T \exp(q_i^\top k_m)}.$$

The updated token representation is

$$s_i = \sum_{j=1}^T \alpha_{ij} v_j.$$

Since all token representations are computed simultaneously, transformers enable full parallelization across sequence positions.

5.4 Multi-Head and Masked Attention

A single attention head may be insufficient to capture diverse relationships within a sequence. Multi-head attention addresses this by using multiple sets of projection matrices (W_Q^i, W_K^i, W_V^i), allowing the model to attend to different aspects of the input simultaneously. In autoregressive settings, masked attention is applied to prevent tokens from attending to future positions. Masking is implemented by assigning negative infinity to invalid positions before the softmax operation, ensuring causal sequence modeling.

5.5 Autoregressive Language Modeling

Transformer-based language models are trained using an autoregressive objective, where the joint probability of a sequence is factorized as

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1}).$$

Each token is predicted using only previously observed tokens, making this formulation suitable for text generation tasks.

5.6 Generative Pre-trained Transformers

GPT models are autoregressive transformer-based language models that predict the most probable next token given prior context. Their effectiveness arises from two key factors: generative pretraining on large-scale unlabeled data and a transformer architecture that enables parallel processing of input sequences. Through pretraining and subsequent fine-tuning, GPT models learn rich contextual representations that can be adapted to a wide range of downstream tasks, including text generation, summarization, and domain-specific analysis.

6 Legal Language and Domain Characteristics

Legal language in this pipeline is treated as structured Q&A pairs where precision, role clarity (system/user/assistant), and consistent formatting matter for stable supervision. The script enforces this by converting a raw `merged.json` into `{instruction, input, output}` and then wrapping it with Llama 3.1's chat headers (`<begin_of_text>`,

`<start_header_id>system|user|assistant)` to preserve speaker turns and legal reasoning context. This helps the model learn normative phrasing (“Answer the following legal question accurately”) and cite-style completions typical of statutes/case questions, while keeping responses bounded by EOS. The approach suits statute interpretation and doctrinal queries where fidelity and role alignment are critical.

6.1 Nature of Legal Text

The code frames legal text as high-stakes, instruction-following sequences: a system directive establishing accuracy, a user legal question, and an assistant answer. This mirrors formal exchanges common in legal practice, emphasizing definitions, scope, and exceptions. By templating prompts with explicit headers and EOS handling, the trainer discourages digressions and promotes determinate, context-aware answers. The LoRA-adapted, 4-bit Llama 3.1 backbone balances capacity and efficiency, letting you fit domain patterns (e.g., Articles, clauses) under modest hardware. As a result, the model learns concise, citation-ready outputs aligned with the Q&A format characteristic of bar-style and statute-interpretation tasks.

6.2 Challenges in Legal NLP

Key challenges addressed by this setup include long-range dependencies (handled via a 2,048-token window), rigid role structure (enforced by the chat template), and limited compute (mitigated by 4-bit quantization and LoRA). The script also combats label noise by explicitly mapping question → answer and fixing a single system instruction, reducing prompt variance. However, remaining hurdles include potential truncation of multi-article contexts, ambiguity in statutory cross-references, and hallucinations in low-evidence scenarios. Post-training evaluation with curated test prompts—and stricter decoding or retrieval augmentation—can further strengthen factuality and scope control beyond the current supervised fine-tuning loop.

7 Dataset Preparation and Analysis

The pipeline reads `merged.json`, transforms each entry to `{instruction, input, output}`, and writes a clean `train.json`. This normalization ensures the trainer consumes uniformly structured samples

and the chat template deterministically renders system/user/assistant segments. Logging of total sample counts offers a minimal health check before training. The choice of a single, domain-anchored system instruction (“Answer the following legal question accurately”) standardizes supervision signals across examples, reducing noise from varied phrasings. A follow-up data audit should examine class balance (topics, statutes), answer lengths, and leakage across train/test to ensure robust generalization for legal reasoning tasks.

7.1 Dataset Identification

In this workflow, the identified dataset is whatever lives in `/content/merged.json`—implicitly a legal Q&A corpus. Each record must expose `question` and `answer`, which the script maps to `input` and `output` while injecting a stable `instruction`. This design assumes a supervised format appropriate for SFT rather than preference or RL signals. Before training, validate sources (e.g., statutory text vs. secondary summaries), jurisdiction coverage, and date ranges to avoid doctrinal drift. If spanning multiple legal systems, consider tagging jurisdiction to guide the model’s context. The final `train.json` becomes the single source of truth consumed by `datasets.load_dataset`.

7.2 Dataset Structuring

Structuring happens in two stages. First, *normalization*: the script converts raw entries to `{instruction, input, output}` and saves them as `train.json`. Second, *prompt rendering*: `formatting_func` wraps each example with the Llama 3.1 chat scaffolding—system → user → assistant—and stores the fully rendered text in a `text` field for the trainer. This explicit rendering prevents template drift and ensures the model learns to place answers only after the assistant header. The `SFTTrainer` then reads `dataset_text_field = “text”`, disabling packing to keep one sample per sequence, which is safer for instruction alignment in legal Q&A.

7.3 Data Quality Considerations

Data quality hinges on faithful question → answer mappings and consistent granularity. The fixed system instruction reduces prompt variability; however, verify that answers are authoritative (statute text, controlling case law) and current per jurisdiction. Watch for leakage (identical or near-duplicate

prompts across splits), over-short answers that omit reasoning, and overly long passages that risk truncation at 2,048 tokens. Consider adding rationale fields to encourage step-wise legal argumentation if desired. Finally, log length distributions and run spot checks before training; even simple counts printed by the script are a start toward dataset sanity verification.

8 Tokenization and Input Processing for LLMs

The code relies on the tokenizer returned alongside the Unsloth-wrapped Llama 3.1 8B model, ensuring byte-pair/subword tokenization consistent with the base checkpoint. It crafts role tokens via literal chat headers and then tokenizes the fully rendered string, maintaining deterministic boundaries between system, user, and assistant. EOS handling is respected during training and at inference, where the script locates the assistant segment and truncates at the first EOS token to isolate the answer. This tight coupling of rendering, tokenization, and generation curbs spillover text and stabilizes legal answer extraction.

8.1 Subword Tokenization

Subword tokenization (implemented by the Llama-family tokenizer) compresses legal vocabulary into frequent sub-units, efficiently handling rare statute names and citations. Because the pipeline formats complete conversations first and tokenizes second, contiguous spans like “Article 124” or “clause (4)” are preserved in context before segmentation. This reduces fragmentation across lines and maintains predictable position of legal markers around parentheses and numerals. The approach also mitigates OOV issues by decomposing rare tokens into known subwords, while keeping sequence lengths manageable for the 2,048-token window used during training and generation.

8.2 Tokenizer–Model Alignment

Alignment is ensured by sourcing model and tokenizer from the same Unsloth Llama 3.1 checkpoint, so the tokenizer vocab, special tokens (including EOS), and chat conventions match the quantized weights. The rendering function hard-codes header strings consistent with that template, preventing role drift. At inference, decoding with the same tokenizer and explicitly searching for the assistant segment and EOS guards against contaminating the extracted answer with trailing tokens. Such strict alignment is

crucial for legal responses where mixing roles or misplacing EOS can yield malformed, potentially misleading outputs. Saving both model and tokenizer together preserves this contract.

8.3 Context Length and Sequence Truncation

The `max_seq_length = 2048` parameter caps how much statute context and dialogue fit into a single example. The trainer consumes fully rendered strings without packing, lowering the risk of inter-sample bleed but increasing sensitivity to overlong prompts. At inference, `max_new_tokens` bounds the answer length, while EOS-aware post-processing further trims output. For longer authorities (e.g., multi-clause articles), consider chunking prompts or retrieval augmentation rather than naive inclusion to avoid truncation. Monitoring token length histograms pre-training and adjusting `max_seq_length` (hardware permitting) can reduce information loss and preserve key legal qualifiers within the window.

9 Fine-Tuning LLMs

Fine-tuning is the process of taking a pretrained machine learning model and further training it on a smaller, targeted data set. The aim of fine-tuning is to maintain the original capabilities of a pretrained model while adapting it to suit more specialized use cases.

During fine-tuning, the model’s parameters are updated using supervised or preference-based learning on curated datasets, such as question–answer pairs, domain-specific documents, or labeled instructions. This enables the model to align its outputs with desired formats, terminology, and task objectives, such as legal document analysis.

Fine-tuning can be broadly classified into 2 categories:

- Full Fine-Tuning
- Parameter-Efficient Fine-Tuning(PEFT)

9.1 Full Fine-Tuning

Full fine-tuning is a training approach in which all parameters of a pre-trained Large Language Model (LLM) are updated using a task or domain-specific dataset. This makes significant changes in the behavior of the model and makes it highly effective for tasks that differ substantially from the original training distribution of the model.

However, since all the parameters of the model are affected, other factors like training time, computing cost and overfitting are all negatively impacted. Therefore, in our project using **Parameter-Efficient Fine-Tuning(PEFT)** is a more effective and a viable technique since it can provide a similar result for a much lower cost and training time.

9.2 Parameter-Efficient Fine-Tuning

PEFT methods introduce or modify a small number of additional trainable parameters while keeping the original model weights frozen.

Common PEFT approaches include:

- **Adapters**
- **Prompt-Tuning**
- **Low-Rank Adaption(LoRA)**
- **Quantized Low-Rank Adaption(QLoRA)**

9.2.1 Adapters

This method involves adding extra layers to the existing transformer model and fine-tuning only these layers, which has been shown to yield results comparable to full model fine-tuning. The adapted layers reduce the input to a smaller dimension, apply a non-linear activation function, and then expand it back to the input dimension, which ensures compatibility with the subsequent transformer layer.

9.2.2 Prompt-Tuning

Here a pre-trained Large Language Model (LLM) is adapted to a specific task by learning a small set of continuous prompt embeddings, while keeping all original model parameters frozen. Instead of modifying the model's weights, prompt tuning optimizes these trainable prompt vectors that are prepended to the model's input.

Prompt tuning uses soft prompts that are not human-readable but are optimized through gradient-based learning.

9.2.3 Low-Rank Adaption(LoRA)

Low-rank adaptation (LoRA) is a technique which can adapt large models to specific uses by adding lightweight pieces to the original model rather than changing the entire model.

Like Prompt-tuning, LoRA also **freezes the weights of the model** and then introduces a **low rank matrix**. This new matrix is adjusted on the basis of the inputs to give us the desired outputs. LoRA

focuses on modifying a smaller subset of parameters (lower-rank matrices) to reduce computational and memory overhead.

For Example, A 4×4 matrix can be decomposed into a 4×1 and a 1×4 matrix. The low-rank matrices are updated through gradient descent during fine-tuning, without modifying the weights of the base model. These matrices contain new weights to apply to the model when generating results. The multiplied change matrix is added to the base model weights to get the final fine-tuned model. This process alters the outputs that the model produces with minimal computing power and training time.

Advantages of LoRA

- A base model can be shared and used to build many small LoRA modules for new tasks
- LoRA makes training more efficient and lowers the hardware barrier to entry because users do not need to calculate the gradients for most parameters
- The linear design of LoRA allows us to merge the trainable matrices with the frozen pre-trained model weights

An extension of LoRA known as **Quantized Low-Rank Adaption(QLoRA)** is used.

9.2.4 Quantized Low-Rank Adaption(QLoRA)

QLoRA is the extended version of LoRA which works by quantizing the precision of the weight parameters in the pre trained LLM to 4-bit precision. NF4 is information-theoretically optimal for data that has a normal distribution, which is a common feature of neural network weights and can represent these weights more accurately than a standard 4-bit float would, within the given bit constraint.

Some innovations which QLoRA introduces are:

4-Bit NormalFloat This datatype is designed specifically for AI applications. Quantizes the weights of neural networks to significantly reduce the memory footprints of models and also maintain performance.

4-bit NormalFloat Quantization is a method designed to efficiently quantize the weights of neural networks into a 4-bit format.

In NF4 datatype we have 4 bits for quantization, so we have 16 distinct values

The weights of pretrained neural networks are assumed to follow a zero-centered normal distribution, meaning they are distributed around a central value

of zero. This normalization is achieved by dividing each weight by absolute maximum value

To quantize a normal distribution into 4-bit datatype we have 16 different values(bins) and to find these values we split the normal distribution into 16 pieces of equal width.

Double Quantization In addition to quantizing the model weights, the scale values are also quantized. This significantly reduces the memory required to store quantization metadata.

10 Implementation

10.1 Unsloth Library

Unsloth is an optimization-focused library designed to enable efficient fine-tuning and inference of large language models (LLMs) on consumer-grade hardware. It provides highly optimized implementations of popular transformer architectures, significantly reducing GPU memory usage and training time while preserving model performance.

A key feature of Unsloth is its seamless integration with parameter-efficient fine-tuning techniques such as Low-Rank Adaptation (LoRA) and Quantized Low-Rank Adaptation (QLoRA). By combining 4-bit quantization with low-rank updates, Unsloth allows large models, including LLaMA-family models, to be fine-tuned using limited computational resources.

Unsloth achieves these improvements through several optimizations:

- Custom CUDA kernels for faster attention and matrix operations
- Memory-efficient 4-bit NormalFloat (NF4) quantization
- Optimized backpropagation with reduced activation storage
- Native support for LoRA and QLoRA adapters

In this project, Unsloth was used to fine-tune a LLaMA 3.1 based model for legal question-answering tasks. The library enabled stable training with a maximum sequence length of 2048 tokens while maintaining low VRAM consumption. This made it possible to train domain-specific legal models efficiently without requiring enterprise-scale GPUs.

Overall, Unsloth plays a critical role in making large-scale legal language model fine-tuning practical, accessible, and cost-effective, aligning well with the objectives of the *Judge It Well* project.

10.2 Hugging Face Integration

Unsloth integrates directly with the Hugging Face ecosystem, allowing the use of pretrained models, tokenizers, and datasets from the Hugging Face Hub with minimal changes to existing workflows. Models are loaded using standard Hugging Face identifiers, while Unsloth internally applies memory and speed optimizations without altering the model architecture.

Tokenization and dataset handling are performed using Hugging Face’s **transformers** and **datasets** libraries, ensuring consistency between pretraining and fine-tuning. For training, Unsloth works seamlessly with Hugging Face training APIs, including supervised fine-tuning setups with LoRA and QLoRA.

After training, fine-tuned models and adapter weights can be saved and deployed using standard Hugging Face formats, enabling easy sharing, inference, and integration into downstream applications. This makes Unsloth an efficient acceleration layer within the Hugging Face workflow.

11 Final Model

11.1 Base Model Selection

In this project, the LLaMA 3.1 (8B) model was selected as the base language model due to its strong instruction-following capability and efficient transformer architecture. The model provides a good balance between representational power and computational feasibility, making it suitable for domain-specific fine-tuning.

LLaMA 3.1 offers improved reasoning performance, long-context support, and compatibility with parameter-efficient fine-tuning techniques such as LoRA and QLoRA. Its open availability and seamless integration with libraries like Hugging Face and Unsloth further enable efficient training and deployment within limited hardware resources.

11.2 Legal Domain Adaptation Strategy

The Judge-It-Well model is adapted to the legal domain using instruction-based supervision and structured legal prompts. Since legal texts follow formal language and task-driven reasoning patterns, direct use of a general-purpose language model is insufficient.

Instruction-Based Supervision Training samples are framed as explicit legal instructions paired with expected responses. This guides the model to follow task-specific directives such as legal explanation, clause interpretation, and summarization, improving consistency and clarity in legal outputs.

Structured Legal Prompts Prompts are designed to clearly separate the legal context and the task objective. This structure helps the model focus on relevant legal information and generate responses aligned with legal language conventions and reasoning styles.

11.3 Inference Configuration

After fine-tuning, the Judge-It-Well model is explicitly prepared for inference to ensure efficient and stable generation of legal responses. The Unsloth framework provides a dedicated inference mode that optimizes execution for large language models.

11.4 Generation and Output Handling

Legal responses are generated autoregressively using controlled sampling to balance coherence and determinism. The generated token sequence is decoded using the tokenizer, and the assistant’s response is extracted cleanly to avoid prompt spillover.

```
with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=256,
        temperature=0.3,
        top_p=0.9,
        do_sample=True,
        eos_token_id=tokenizer.eos_token_id,
    )

response = tokenizer.decode(outputs[0],
    skip_special_tokens=True)
response = response.split("### Response: ")[-1].strip()
```

11.5 Example Usage of the Final Model

The fine-tuned Judge-It-Well model is queried using legal-style prompts to demonstrate its ability to generate domain-aligned and context-aware legal responses.

```
print(ask_legal_bot(
    "What is the difference between cognizable and non-
    -cognizable offences under CrPC?"
))
```

Sample Output

```
Cognizable offences are those for which the police can
register an FIR
and initiate investigation without prior court
approval, whereas
non-cognizable offences require permission from a
magistrate before
any investigation can be carried out.
```

12 Evaluation and Capabilities

12.1 Qualitative Evaluation

The evaluation of the fine-tuned model focuses on both performance and reliability in legal language tasks. Key aspects include the accuracy of legal question answering, consistency of responses, and adherence to legal context and terminology.

In addition to qualitative assessment, the model is evaluated for generalization across unseen legal queries, robustness to ambiguous inputs, and efficiency in terms of inference speed and memory usage. These criteria ensure that the model is not only accurate but also practical for real-world legal applications

12.2 Capabilities

After domain adaptation, Judge-It-Well demonstrates the ability to handle several legal text understanding tasks. The model can answer legal questions using appropriate terminology, explain legal distinctions in a structured manner, and summarize legal text while preserving essential context. These capabilities indicate improved alignment with legal reasoning patterns compared to a general-purpose language model.

12.3 Limitations

Despite domain-specific fine-tuning, Judge-It-Well does not possess authoritative legal reasoning or decision-making capability. The model may generate responses that appear legally plausible but are incomplete or inaccurate. As a result, its outputs should be treated as assistive explanations and must be verified by individuals with legal expertise. The model is not intended to replace professional legal judgment.

13 Conclusion and Future Scope

This project presents the development of *Judge-It-Well*, a legal-domain language model built by progressively advancing from foundational machine learning concepts to transformer-based large language models. Through this journey, theoretical understanding of neural networks, sequence models, and attention mechanisms is translated into a practical system capable of interpreting and reasoning over le-

gal text.

A key outcome of this project is the effective use of efficient fine-tuning techniques to adapt a large pre-trained language model to the legal domain. By leveraging parameter-efficient adaptation and the Unsloth framework, domain alignment is achieved without the computational cost of full model retraining. Future work may extend this system through larger and more diverse legal datasets, retrieval-augmented generation for handling long legal documents, and systematic evaluation using legal-domain benchmarks.